

Series: STIG Hardening Docker Containers & Applications (Part 1 of 6)

Author: H. Brister Wooley

Published: 2026-03-15

Series: STIG Hardening Docker Containers & Applications (1/6)

Published Link: <https://github.com/hbwooley/deep-thoughts-and-ramblings/blob/main/2026-03-15-stig-hardening-docker-part1.md>

Grab a coffee. Actually, grab two – you’ll need them.

You’ve heard the phrase: “We’re STIG compliant.” Maybe you’ve even said it with a straight face. Then the auditor shows up, asks for the `sshd_config` inside your container, and you realize... there *is* no `sshd_config`. Because it’s a container. And the STIG was written for VMs back when “cloud” meant “someone else’s server room.”

Welcome to the weird world of STIG-hardening Docker.

I’ve been through this rodeo. Multiple times. The conversation always goes the same way:

Auditor: “Show me the SSH configuration.”

You: “We don’t run SSH in containers.”

Auditor: “The STIG requires SSH hardening.”

You: “The STIG was written in 2015. For VMs.”

Auditor: “Findings are findings. Document the compensating control.”

And just like that, you’re writing a waiver for a control that literally cannot apply to your architecture. This is the reality of STIG in 2026 – and it’s exactly why most teams fail.

This 6-part series walks through taking a containerized application – from base image to runtime to app code – and making it pass a STIG audit *without* lying on the paperwork. We’ll use real controls from the **Docker Enterprise 2.x Linux/UNIX STIG** (V-235775 through V-235777) and the **Apache Server 2.4 UNIX Server STIG** (V-214228 through V-214274) – the exact controls from a production DoD assessment.

The STIG Framework: What You're Actually Up Against

The Security Technical Implementation Guide (STIG) isn't a suggestion. It's a set of mandatory configuration standards for DoD systems. Each control has a **V-ID** (Vulnerability ID), a **Rule ID**, a **Severity** (CAT I/II/III), a **CCI Reference** (mapping to NIST 800-53), and a **Check** – the specific test the auditor runs.

From your Apache STIG:

```
V-214246 | SV-214246r960759_rule | CAT II
STIG ID: AS24-U1-000360
CCI: CCI-000366
Title: The Apache web server must listen on a specific IP address and port.
Check: Verify the Listen directive is configured to a specific IP:port.
Fix: Configure Listen 192.0.2.10:80
```

From your Docker STIG:

```
V-235777 | SV-235777r627458_rule | CAT I
STIG ID: DKER-EE-001070
CCI: CCI-000015, CCI-000381, CCI-000382, CCI-001184, CCI-001762, CCI-002418,
CCI-002420, CCI-002421, CCI-002422
Title: FIPS mode must be enabled on all Docker Engine - Enterprise nodes.
Check: docker info -> "Security Options" section must show "fips" label
Fix: Enable FIPS mode on host OS, or set DOCKER_FIPS=1
```

In a VM world, you edit `/etc/httpd/conf/httpd.conf`, run the check, pass. In a container world, that config is baked into an image, mounted via volume, or generated at startup. The auditor doesn't care *how* it gets there – they care that the check passes *and* you can prove it.

Why Docker Makes STIG Weird (And Why That's Your Problem)

Here's the fundamental mismatch: the STIG assumes a world that hasn't existed since 2015.

STIG Assumption	Container Reality	STIG Controls Affected
Persistent filesystem	Immutable layers + tmpfs + volumes	V-214256 (file permissions), V-214231 (audit logging)
Single OS instance	Shared kernel, isolated userspace	V-235777 (FIPS), V-214230 (crypto)
systemd manages services	PID 1 is your app (or tini/dumb-init)	V-214230 (process management)
Config files in /etc	Configs baked, mounted, or env-substituted	V-214256 (config protection)

STIG Assumption	Container Reality	STIG Controls Affected
Root access for admin	Root dropped at build <i>and</i> runtime	V-214246 (least privilege), V-214230 (no escalation)
Network interfaces persistent	Ephemeral IPs, overlay networks, service mesh	V-214246 (specific IP binding)
sshd for remote access	No SSH – API via Docker socket or orchestrator	V-235776 (TCP socket binding)

The STIG doesn't know about `cap_drop`, `read_only: true`, or `security_opt: no-new-privileges`. But those are *how* you satisfy controls like “prevent privilege escalation” (V-214230) or “run with least privilege” (V-214246) in a container.

Here's the thing: The controls don't change. The *implementation* does. Your job is to map the control to the container-native equivalent and prove it works.

The Three STIGs You're Actually Dealing With

Your project has **three STIGs** to satisfy – and yes, they overlap in ways that will make you question your career choices:

1. Docker Enterprise 2.x Linux/UNIX STIG (DKER-EE)

- **Scope:** Docker daemon, UCP, DTR, Swarm Mode
- **Key Controls:**
 - V-235775: Per-user session limits in UCP (CAT II)
 - V-235776: Disable TCP socket binding (CAT II)
 - V-235777: FIPS mode enabled (CAT I) – *the one that keeps you up at night*
 - V-235778+: Container runtime, image signing, registry auth

2. Apache Server 2.4 UNIX Server STIG (AS24-U1)

- **Scope:** Apache HTTPD 2.4 configuration
- **Key Controls** (47 controls, V-214228 through V-214274):
 - V-214228: KeepAlive On, MaxKeepAliveRequests >= 100
 - V-214229: Session modules loaded (session_module, usertrack_module)
 - V-214230: TLS 1.2 only, FIPS-approved crypto
 - V-214231: Audit logging with specific fields
 - V-214232/V-214233: Log format and retention
 - V-214246: Listen on specific IP (not 0.0.0.0)
 - V-214256: File permissions, directory restrictions
 - V-214260+: ModSecurity, rate limiting, DoS protection

3. PostgreSQL 13 STIG

- **Scope:** Database configuration
 - **Key Controls:** Authentication, encryption, auditing, permissions
-

The Mindset Shift: Checklist -> Evidence

Old mindset: “I checked the box. The config is in the image. We’re good.”

STIG mindset: “Show me the running container. Show me the effective config. Show me the test that proves it. Show me the pipeline that rebuilds it when the STIG updates.”

Every control needs **three artifacts** – and if you’re missing any of these, you don’t have compliance, you have hope:

Artifact	What It Looks Like	Example for V-214246
Implementation	Dockerfile directive, compose setting, app code	Listen 192.0.2.10:80 in httpd.conf (baked via COPY)
Verification	Command that proves it works <i>in the running container</i>	<code>docker exec <ctr> netstat -tlnp grep :80 -> 192.0.2.10:80</code>
Automation	CI/CD step that fails the build if it regresses	CI test: <code>curl -H "Host: evil.com" https://192.0.2.10/ -> 403</code>

The CCI Connection: Why It Matters

Every STIG control maps to **CCI (Common Configuration Identifiers)** which map to **NIST 800-53 controls**. This is your traceability chain – and your lifeline when the auditor asks “why?”:

```
STIG V-214246 (AS24-U1-000360)
-> CCI-000366
-> NIST 800-53 CM-6 (Configuration Settings)
-> NIST 800-53 SC-7 (Boundary Protection)
```

When the auditor asks “why this control?”, you show the chain. When NIST 800-53 Rev 5 drops, you know exactly which STIG controls need updating. This isn’t bureaucracy – it’s your defense.

What This Means for Your Strategy

Most teams treat STIG as a **gate** – something to pass before deployment. That’s backwards.

STIG is a **continuous property** of your system. The moment you deploy, drift starts. A base image update, a dependency upgrade, a developer's "quick fix" – any of them can invalidate a control.

The strategy that works: **build the evidence generation into the pipeline**. Every commit proves compliance. Every deploy re-verifies. The audit becomes "show me the last 30 days of passing tests" – not "let me log into production and check."

What's Coming Next

Part 2: Base image selection. Why Alpine 3.20 beats Ubuntu 24.04 for STIG (smaller attack surface, fewer packages to justify), why distroless is the endgame, and the exact packages you *must* remove to satisfy V-214230 (privilege escalation prevention).

Hint: it's sudo. And su. And cron. And busybox-suid (which gives you SUID su out of the box – CAT I finding on day one).

[Read Part 2 ->](#)

Appendix: CompTIA Security+ SY0-701 CEU Mapping

This appendix maps each technique in this post to CompTIA Security+ exam objectives for CEU submission.

Technique Implemented	Security+ Domain	SY0-701 Objective	Rationale
STIG-to-container-native mapping	Domain 3: Security Architecture	3.2: Implement secure system design	Translating compliance controls to container primitives
Implementation/Verification/Automation triad	Domain 4: Security Operations	4.5: Implement automation/orchestration	Continuous compliance validation pipeline
CCI -> NIST 800-53 traceability	Domain 1: Threats/Vulns/Mitigations	1.2: Explain threat modeling concepts	Regulatory framework mapping for risk assessment
Immutable infrastructure for config drift	Domain 3: Security Architecture	3.2: Implement secure system design	Prevents configuration drift via image-based deployment
Evidence-based compliance (not checkbox)	Domain 4: Security Operations	4.1: Apply security monitoring concepts	Audit-ready evidence generation
Docker native controls (cap_drop, read_only)	Domain 3: Security Architecture	3.2: Implement secure system design	Least privilege via kernel-enforced capabilities

Primary Domain Coverage: Domain 3 (Architecture & Engineering) – ~45% | Domain 4 (Operations) – ~30% | Domain 1 (Threats) – ~25%

Total Security+ Mapping: >=50% of content maps to SY0-701 objectives